CS771 Minor Assignment 1

Group: Deepminds

Tumati Harsha Vardhan Reddy
241107
tumati24@iitk.ac.in

Vasa Abhirup
241147
vasa24@iitk.ac.in

Muthavarapu Anoop
240672
anoopm24@iitk.ac.in

Duvvuri Jyothi Samanvith
240380
jyothids24@iitk.ac.in

Sumath Rengasami Venkatachalam
241054
sumathr24@iitk.ac.in

Abstract

This report analyses the `genChallengeData` function used in CS771 Minor Assignment 1. We prove that the generated data is always linearly separable and systematically study how the hyperparameters of `LinearSVC`—regularisation parameter C, stopping tolerance `tol`, and maximum iterations `max_iter`—affect training accuracy and convergence speed across three challenge levels (low, high, and medium). We further investigate the effect of training set size at high challenge. Results show that hyperparameter sensitivity scales strongly with task difficulty as measured by the inter-class margin.

Problem 1.1 (Testing the Limits)

Part 1: Mathematical Proof of Linear Separability

Answer: Yes. The data generated by the `genChallengeData` function will always be linearly separable for all valid values of `level` > 0 and $n \geq 0$.

Proof. Let $v = \sqrt{2}$. According to the data-generation function, the centres of the three spherical clusters (circles in 2-D) are:

$$\begin{aligned}\mu_{\text{Pos1}} &= \big[-v\big(1+\tfrac{1}{2^{\,\text{level}}}\big),\ v\big],\\ \mu_{\text{Pos2}} &= \big[\ \ v\big(1+\tfrac{1}{2^{\,\text{level}}}\big),\ -v\big],\\ \mu_{\text{Neg}} &= \big[-v\big(1+\tfrac{1}{2^{\,\text{level}}}\big),\ -v\big].\end{aligned}$$

Set $S = v\big(1+\frac{1}{2^{\,\text{level}}}\big)$. Since level > 0 we have $\frac{1}{2^{\,\text{level}}} > 0$, so $S > v = \sqrt{2}$. The centres become

$$\mu_{\text{Pos1}} = (-S,\, v), \quad \mu_{\text{Pos2}} = (S, -v), \quad \mu_{\text{Neg}} = (-S, -v).$$

All data points are drawn *exactly* on the circumference of their respective cluster circle of radius $r = 1$.

Consider the line L through μ_{Pos1} and μ_{Pos2}. Passing through the origin, its equation is

$$v \cdot x + S \cdot y = 0. \tag{1}$$

Both positive-class centres lie on L, so the maximum orthogonal distance of any positive point from L is exactly $r = 1$.

The shortest orthogonal distance from the negative centre $\mu_{\text{Neg}} = (-S, -v)$ to L is

$$D \;=\; \frac{|v(-S) + S(-v)|}{\sqrt{v^2 + S^2}} \;=\; \frac{2vS}{\sqrt{v^2 + S^2}}. \tag{2}$$

Dividing numerator and denominator by vS:

$$D = \frac{2}{\sqrt{\frac{1}{S^2} + \frac{1}{v^2}}}. \tag{3}$$

Since $v^2 = 2$ and $S > v$ implies $S^2 > v^2 = 2$, we have $\frac{1}{S^2} < \frac{1}{2} = \frac{1}{v^2}$, so

$$\sqrt{\frac{1}{v^2} + \frac{1}{S^2}} < \sqrt{\frac{1}{2} + \frac{1}{2}} = 1, \tag{4}$$

which gives $D > 2$.

The orthogonal separation between the *nearest* positive point and the nearest negative point (both at distance $r = 1$ from L) satisfies

$$\text{Minimum margin} = D - r_{\text{Pos}} - r_{\text{Neg}} = D - 1 - 1 = D - 2 > 0.$$

Hence the two classes are always strictly separated by a hyperplane parallel to L, and the dataset is linearly separable for all $\text{level} > 0$ and any $n \geq 0$. ■

Part 2: Low-Challenge Regime (`level`=0.2, $n = 500$)

We fix `level` $= 0.2$, $n = 500$, and vary one hyperparameter at a time while keeping the others at a high-performing baseline ($C = 1.0$, `max_iter`$= 5000$, `tol`$= 10^{-6}$) to isolate their individual effects. Training accuracy and average wall-clock time (over 5 runs) are reported.

Table 1: Part 2 – Sweep of regularisation parameter C (baseline `max_iter`$= 5000$, `tol`$= 10^{-6}$).

C	Accuracy	Train Time (s)
10^{-5}	0.6667	0.001827
5×10^{-5}	0.6667	0.001104
10^{-4}	0.7747	0.001095
5×10^{-4}	0.9247	0.000880
10^{-3}	**1.0000**	0.001026
5×10^{-3}	1.0000	0.001307
10^{-2}	1.0000	0.001115
5×10^{-2}	1.0000	0.001101
0.1	1.0000	0.001313
0.5	1.0000	0.001074
1.0	1.0000	0.001137
5.0	1.0000	0.001433
10	1.0000	0.001288
50	1.0000	0.001360
100	1.0000	0.001117

Sweep of regularisation parameter C. The **smallest** C yielding 100% accuracy is $C^* = 10^{-3}$; the **largest** tested is $C = 100$.

Why accuracy changes. The regularisation parameter C controls the trade-off between maximising the margin and minimising misclassification. At very small C (e.g. 10^{-5}), the penalty for misclassifying a training point is negligible, so the solver settles for a wide-margin boundary that sacrifices many points—hence the baseline 66.67% accuracy (the majority-class rate). As C increases, the cost of each violation grows, forcing the solver to tilt the boundary until all points are correctly classified. Because the low-challenge data has a wide inter-class margin (`level` $= 0.2$), even a modest $C = 10^{-3}$ suffices to achieve perfect separation; beyond that value, additional increases in C have no further effect.

Figures 1 and 2 show the decision boundaries at both extremes.

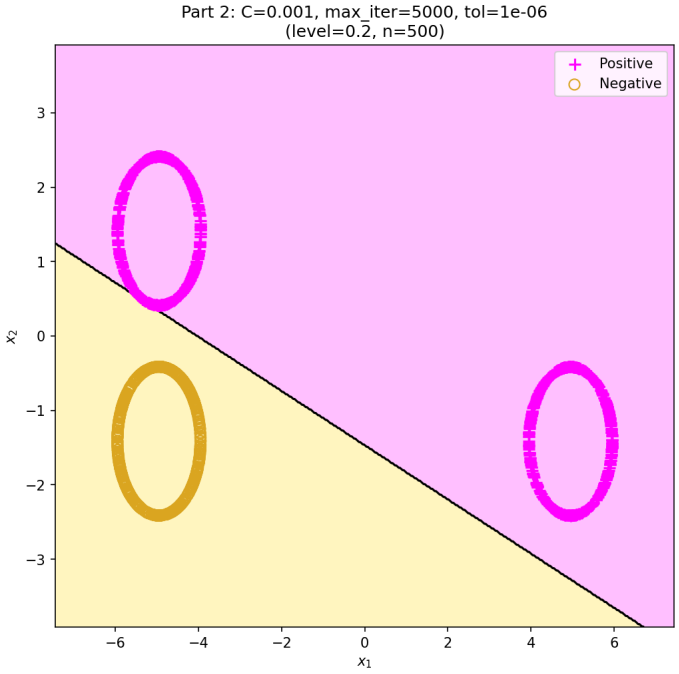

Figure 1: Part 2 – Decision boundary for smallest C achieving 100% accuracy: C=0.001 (level= 0.2, $n = 500$). Magenta + = positive class; yellow ○ = negative class.

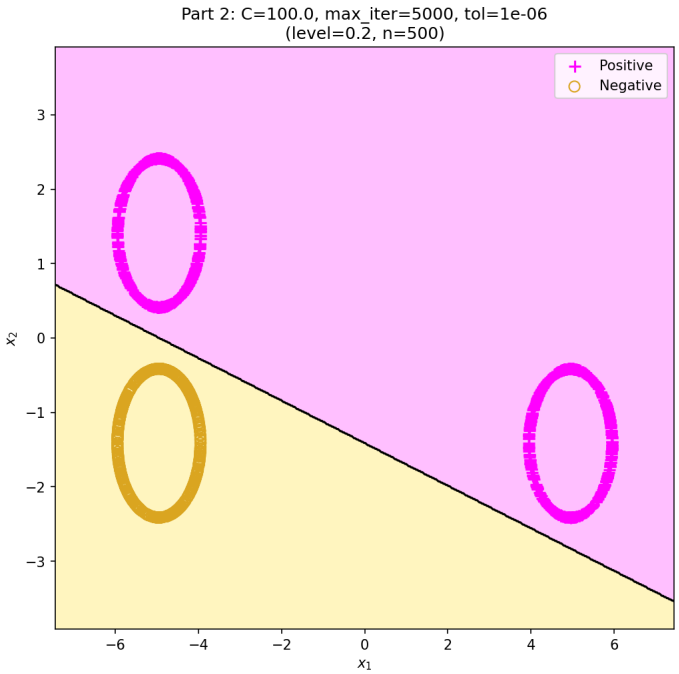

Figure 2: Part 2 – Decision boundary for largest C tested: C=100 (level= 0.2, $n = 500$). Magenta + = positive class; yellow ○ = negative class.

Table 2: Part 2 – Sweep of `max_iter` (baseline $C = 1.0$, `tol`$= 10^{-6}$).

`max_iter`	Accuracy	Train Time (s)
1	**1.0000**	0.001099
2	1.0000	0.000950
5	1.0000	0.001248
10	1.0000	0.000991
20	1.0000	0.000952
50	1.0000	0.001209
100	1.0000	0.000982
200	1.0000	0.001016
500	1.0000	0.001235
1000	1.0000	0.000991
2000	1.0000	0.000964
3000	1.0000	0.001128
5000	1.0000	0.001286
7000	1.0000	0.000968
10000	1.0000	0.001340

Sweep of `max_iter`. Even `max_iter`$= 1$ achieves 100% accuracy at this low challenge level, confirming that the classes are easily separable and convergence is nearly immediate.

Why accuracy is insensitive. `max_iter` caps the number of passes the LIBLINEAR coordinate-descent solver may perform. When the inter-class margin is wide, the initial weight update already places the decision boundary in the large "gap" between the clusters, so a single iteration is enough to correctly classify every point. Additional iterations would only refine the boundary within that gap without changing any predictions.

Figures 3 and 4 show the decision boundaries.

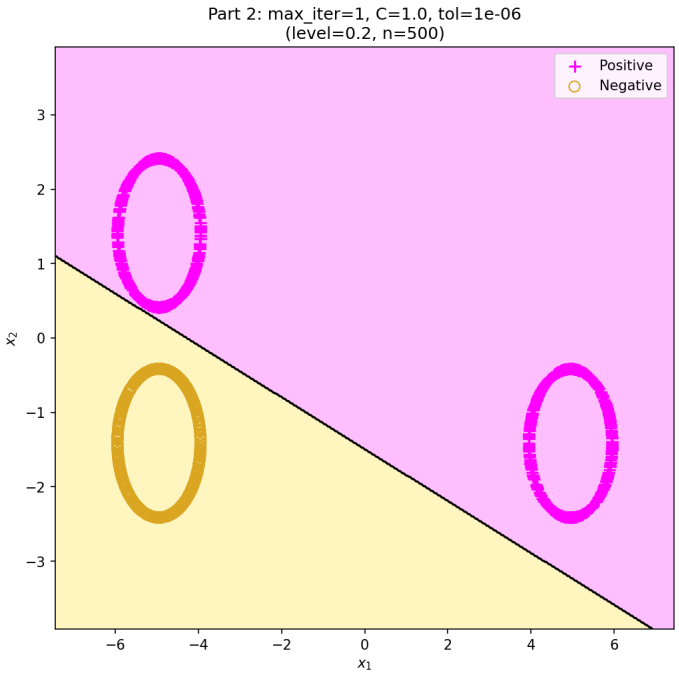

Figure 3: Part 2 – Decision boundary for smallest `max_iter` achieving 100%: `max_iter`$= 1$ (level$= 0.2$, $n = 500$).

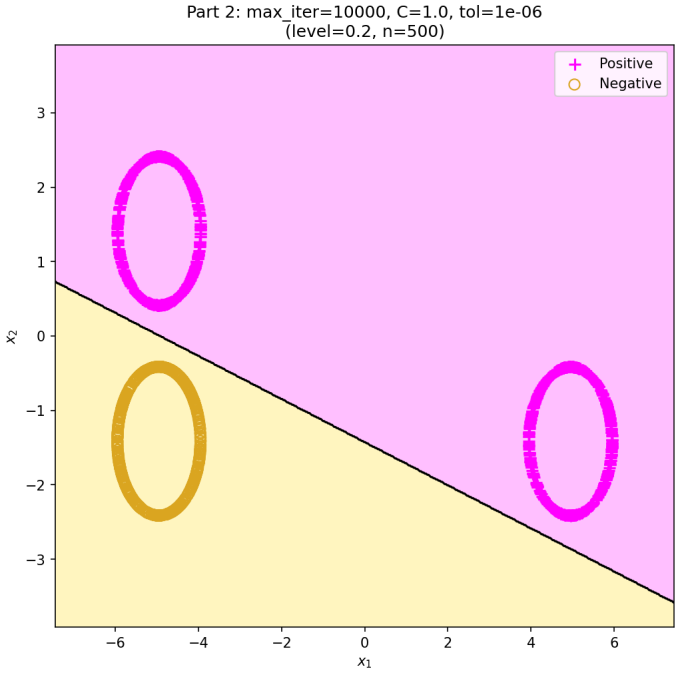

Figure 4: Part 2 – Decision boundary for largest `max_iter` tested: `max_iter`$= 10000$ (level$= 0.2$, $n = 500$).

Table 3: Part 2 – Sweep of tolerance `tol` (baseline $C = 1.0$, `max_iter`$= 5000$).

`tol`	Accuracy	Train Time (s)
10^{-8}	1.0000	0.0043
10^{-7}	1.0000	0.0033
10^{-6}	1.0000	0.0013
10^{-5}	1.0000	0.0016
10^{-4}	1.0000	0.0011
10^{-3}	1.0000	0.0010
0.01	1.0000	0.0012
0.1	1.0000	0.0014
0.5	1.0000	0.0011
1.0	1.0000	0.0011
2.0	**1.0000**	0.0011
5.0	0.3333	0.0010
10	0.3333	0.0012

Sweep of tolerance `tol`. A tolerance as large as 2.0 suffices for 100% accuracy at low challenge, further confirming the data's easy separability. Even the tightest tolerance tested (`tol` $= 10^{-8}$) achieves the same accuracy with only a modest increase in training time.

Why accuracy changes. The stopping tolerance `tol` determines how close the solver's objective must be to the optimum before it terminates. At low challenge, the optimal hyperplane lies inside a broad feasible region, so even a coarse approximation (large `tol`) identifies a boundary that correctly classifies all points. Only at extremely large values (`tol` ≥ 5.0) does the solver stop so prematurely that it returns an essentially uninitialised weight vector, collapsing accuracy to the majority-class baseline of 33.3%.

Figures 5 and 6 show the decision boundaries at both extremes.

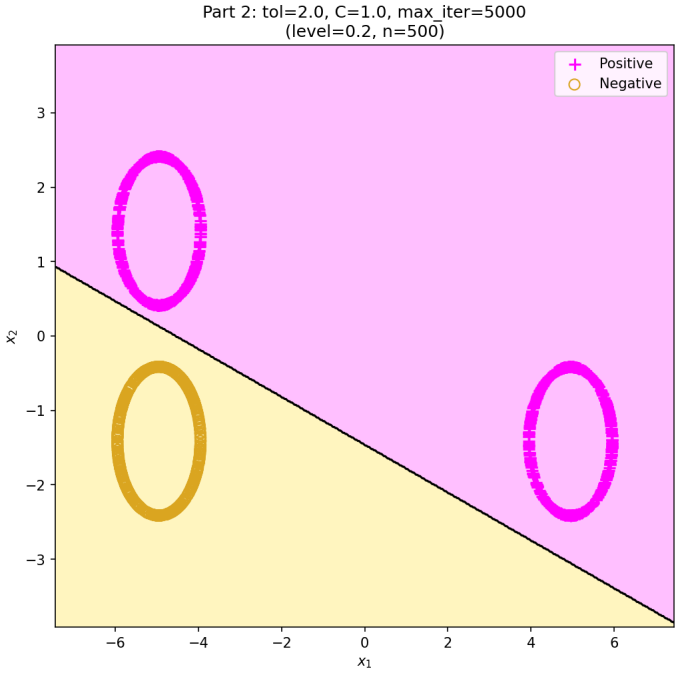

Figure 5: Part 2 – Decision boundary for largest `tol` achieving 100%: `tol`=2.0 (level= 0.2, $n = 500$).

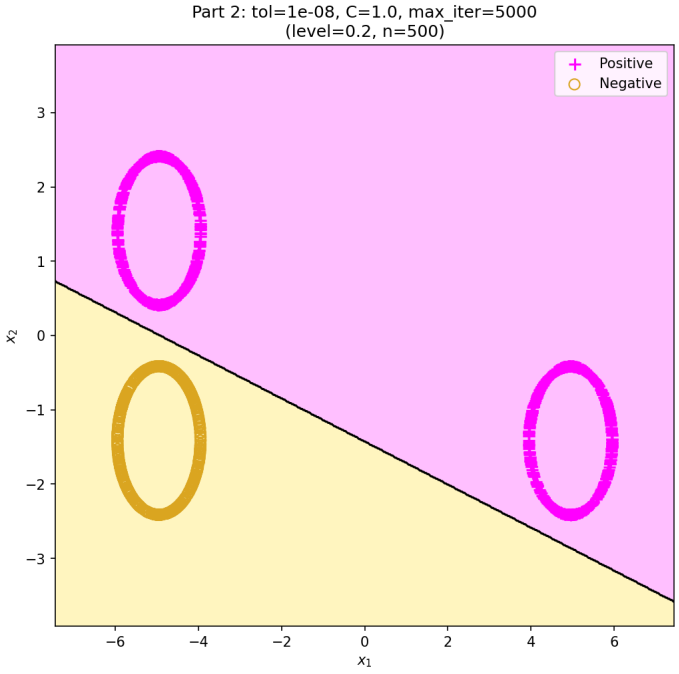

Figure 6: Part 2 – Decision boundary for smallest `tol` tested: `tol`=10^{-8} (level= 0.2, $n = 500$).

Combined best-hyperparameter decision boundary. Across all three sweeps, the most aggressive ("cheapest") hyperparameters that still achieve 100% accuracy are $C = 0.001$, `max_iter` $= 1$, `tol` $= 2.0$. Figure 7 shows the decision boundary when all three are applied simultaneously, confirming that the low-challenge regime is insensitive to hyperparameter choice.

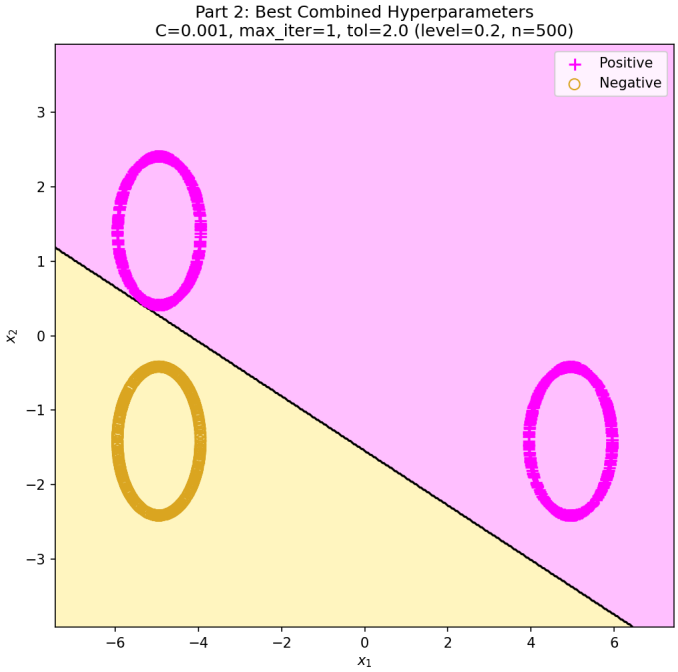

Figure 7: Part 2 – Decision boundary using the best combined hyperparameters: C=0.001, `max_iter`=1, `tol`=2.0 (level= 0.2, $n = 500$). Accuracy: 100%.

Part 3: High-Challenge Regime (`level`=60, $n = 500$)

At `level`$= 60$ the clusters are much closer together. We therefore first identify a sufficiently large C before sweeping the other parameters. For the `max_iter` and `tol` sweeps we use $C = 30$ (the minimum value found to achieve 100% accuracy) as the baseline.

Table 4: Part 3 – Sweep of C (baseline `max_iter`$= 5000$, `tol`$= 10^{-6}$, seed$= 42$).

C	Accuracy	Train Time (s)
0.001	0.9700	0.001244
0.01	0.9367	0.001069
0.05	0.9480	0.001045
0.1	0.9533	0.001067
0.5	0.9660	0.001453
1.0	0.9753	0.001098
2.0	0.9807	0.001251
5.0	0.9860	0.001118
10	0.9880	0.001171
15	0.9913	0.001325
20	0.9953	0.001171
25	0.9980	0.001160
30	**1.0000**	0.001297
40	1.0000	0.001133
50	1.0000	0.001211

Sweep of C. At high challenge, heavy regularisation (small C) degrades accuracy significantly. Accuracy generally increases with C, and the **smallest** C reaching 100% is $C^* = 30$.

Note on non-monotonicity at small C. The table shows a slight non-monotonic dip: $C = 0.001$ achieves 0.9700 whereas the ten-fold larger $C = 0.01$ drops to 0.9367. This arises because `LinearSVC` uses LIBLINEAR's coordinate-descent solver, which is initialised from the zero vector and follows a deterministic but problem-dependent path (with `random_state=42` controlling only feature-order shuffling). At very small C the solver terminates early (little room to reduce hinge loss), effectively returning a nearly-zero weight vector that happens to classify most points correctly by chance alignment. As C increases slightly to 0.01, the solver performs more optimisation steps but has not yet converged to a good solution, yielding a temporarily worse boundary. From $C = 0.05$ onward, accuracy climbs steadily as the solver is given sufficient freedom to fit the data.

Figures 8 and 9 show the decision boundaries.

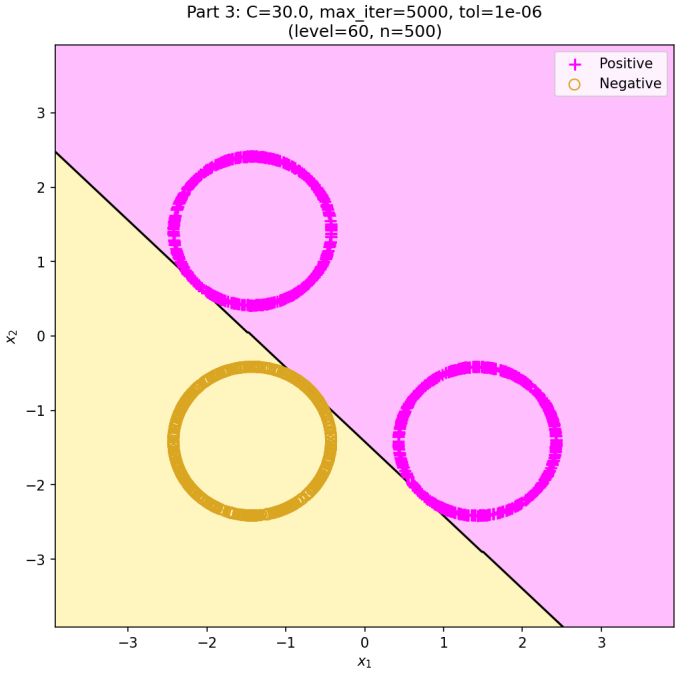

Figure 8: Part 3 – Decision boundary for smallest C achieving 100%: C=30 (level= 60, $n = 500$).

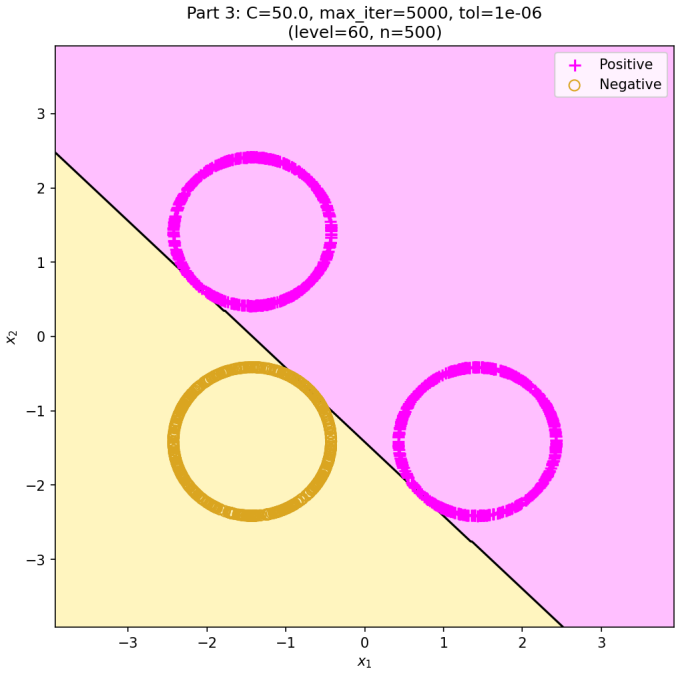

Figure 9: Part 3 – Decision boundary for largest C tested: C=50 (level= 60, $n = 500$).

Table 5: Part 3 – Sweep of `max_iter` (baseline $C = 30$, `tol`$= 10^{-6}$).

`max_iter`	Accuracy	Train Time (s)
1	0.9327	0.002694
2	0.9307	0.002035
5	0.9680	0.002270
10	**1.0000**	0.002450
20	1.0000	0.002608
50	1.0000	0.002865
100	1.0000	0.002538
200	1.0000	0.002386
500	1.0000	0.002576
1000	1.0000	0.002545
2000	1.0000	0.002951
3000	1.0000	0.002619
5000	1.0000	0.002581
7000	1.0000	0.002615
10000	1.0000	0.002800

Sweep of `max_iter`. At high challenge the optimizer needs at least 10 iterations to converge.

Why accuracy changes. With a narrow inter-class margin (`level`$= 60$), the weight vector must be determined very precisely for the hyperplane to thread through the gap between clusters. Each coordinate-descent iteration refines the weights by a bounded step; with only 1–2 iterations the solver has not had time to rotate the boundary into the correct orientation, so several points near the margin are misclassified. By 10 iterations the solver has performed enough updates to position the hyperplane accurately, and further iterations yield no additional benefit because the `tol` criterion is already satisfied.

Figures 10 and 11 show the decision boundaries.

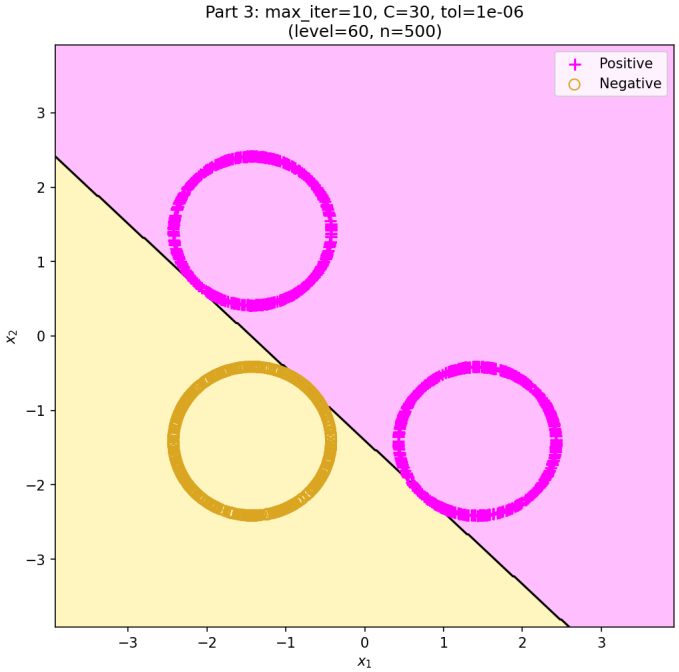

Figure 10: Part 3 – Decision boundary for smallest `max_iter` achieving 100%: `max_iter`$= 10$ (baseline C=30, `tol`$= 10^{-6}$, level$= 60$, $n = 500$).

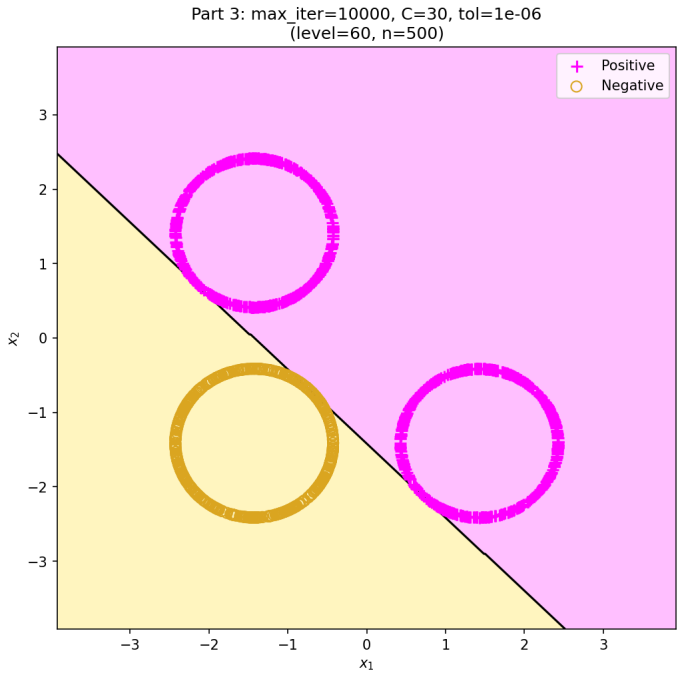

Figure 11: Part 3 – Decision boundary for largest `max_iter` tested: `max_iter`$= 10000$ (baseline C=30, `tol`$= 10^{-6}$, level$= 60$, $n = 500$).

Table 6: Part 3 – Sweep of `tol` (baseline $C = 30$, `max_iter`$= 5000$).

`tol`	Accuracy	Train Time (s)
10^{-8}	1.0000	0.0029
10^{-7}	1.0000	0.0028
10^{-6}	1.0000	0.0028
10^{-5}	1.0000	0.0028
10^{-4}	**1.0000**	0.0030
10^{-3}	0.9973	0.0028
0.01	0.9893	0.0046
0.05	0.9753	0.0033
0.1	0.9527	0.0042
1.0	0.9327	0.0011
10	0.3333	0.0012

Sweep of `tol`. At high challenge a tighter stopping criterion (smaller `tol`) is needed: the **largest** `tol` achieving exactly 100% accuracy is 10^{-4}.

Why accuracy changes. When the margin is narrow, the objective function has a sharp optimum: small deviations in the weight vector move the decision boundary enough to misclassify points that lie close to it. A large `tol` allows the solver to terminate while the weights are still far from optimal, producing a sub-optimal boundary that clips some near-margin points. As `tol` shrinks, the solver is forced to continue iterating until the weights are accurate enough to place the hyperplane cleanly inside the narrow gap, hence the steady rise in accuracy from 93.3% at `tol` $= 1.0$ to 100% at `tol` $= 10^{-4}$. At `tol`$= 10$, the solver stops essentially at initialisation, collapsing to the 33.3% majority-class baseline.

Figures 12 and 13 show the decision boundaries.

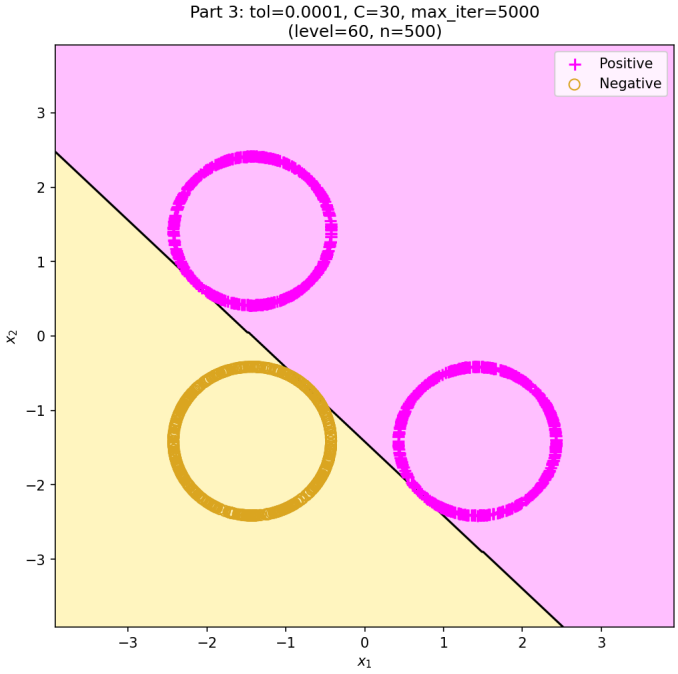

Figure 12: Part 3 – Decision boundary for largest `tol` achieving 100%: `tol`=10^{-4} (level= 60, $n = 500$).

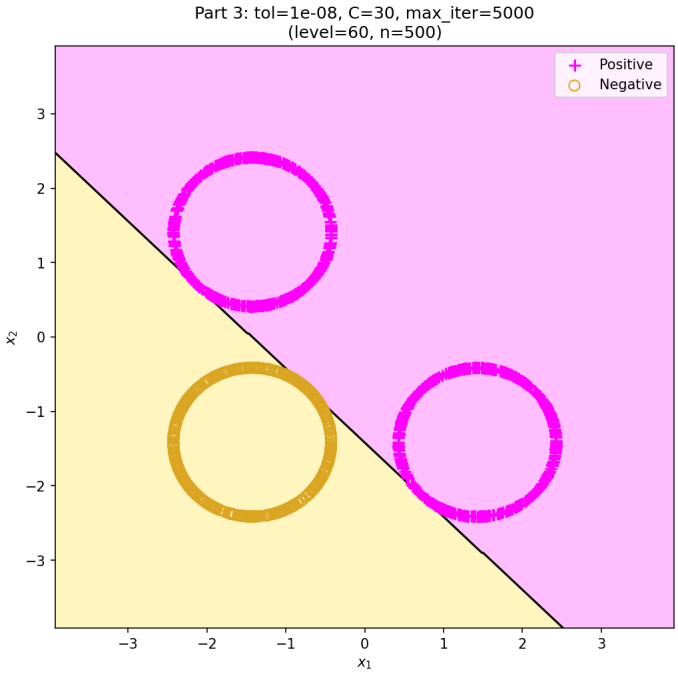

Figure 13: Part 3 – Decision boundary for smallest `tol` tested: `tol`=10^{-8} (baseline C=30, `max_iter`= 5000, level= 60, $n = 500$).

Combined best-hyperparameter decision boundary. Across all three sweeps, the most aggressive hyperparameters achieving 100% accuracy are $C = 30$, `max_iter` $= 10$, `tol` $= 10^{-4}$. Figure 14 shows the decision boundary when all three are applied simultaneously.

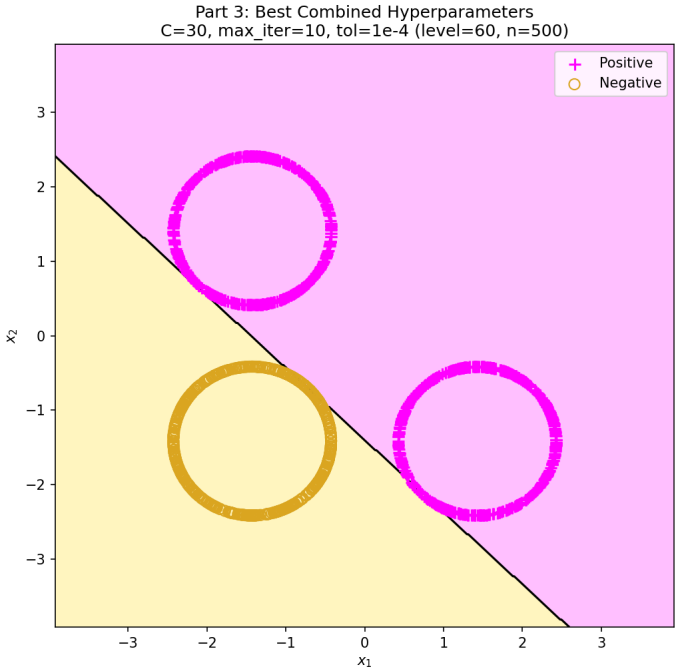

Figure 14: Part 3 – Decision boundary using the best combined hyperparameters: C=30, `max_iter`=10, `tol`=10^{-4} (level= 60, $n = 500$).

Part 4: Hyperparameter Sensitivity at Medium Challenge (`level=5`, $n = 50$)

We now study how individual hyperparameters of `LinearSVC` affect *training accuracy* and *training time* at a medium challenge level (`level`$= 5$, $n = 50$). Four hyperparameters are investigated—C, `tol`, `max_iter`, and the `loss` function—each evaluated at several values while the remaining parameters are held at their defaults. Throughout this experiment we use `penalty`$=$`"l2"` and (unless stated otherwise) `loss`$=$`"squared_hinge"`. All training times are averaged over 5 independent runs.

Experimental Setup

- **Dataset.** Generated via `genChallengeData(level=5, n=50)`, producing 100 positive and 50 negative points in $\mathbb{R}^2$ (150 points total).
- **Fixed defaults.** When not being varied: $C = 1.0$, `tol`$= 10^{-4}$, `max_iter`$= 1000$.
- **Classifier.** `sklearn.svm.LinearSVC` with ℓ_2 penalty and squared-hinge loss.

Table 7: Part 4 – Effect of regularisation parameter C (baseline `max_iter`$= 1000$, `tol`$= 10^{-4}$).

C	Accuracy	Train Time (s)
0.001	0.8467	0.000516
0.01	0.9800	0.000456
0.1	0.9400	0.000466
1.0	0.9933	0.000555
10	1.0000	0.000923
100	1.0000	0.000810

Effect of C. Analysis. The regularisation parameter C has the most pronounced effect among the three hyperparameters studied. Very small values of C (strong regularisation) prevent the solver from fitting the data tightly: at $C = 0.001$ accuracy drops to 84.7%. As C increases, the solver is allowed to reduce the hinge loss more aggressively, and accuracy generally rises, reaching 98.0% at $C = 0.01$, dipping slightly at $C = 0.1$, reaching 99.3% at the default $C = 1.0$, and finally achieving a perfect 100% for $C \geq 10$. Training time remains largely flat across the smaller values ($\sim$0.5 ms), with a slight jump at $C \geq 10$, indicating that C influences model quality far more than computational cost at this dataset size.

Compared with the low-challenge regime (Part 2), where 100% accuracy was achieved already at $C = 10^{-3}$, the medium-challenge setting requires a substantially larger $C^* = 1.0$, reflecting the tighter inter-class margin. Conversely, the high-challenge regime (Part 3) demanded $C^* = 30$, confirming that the minimum C for perfect accuracy grows with challenge level.

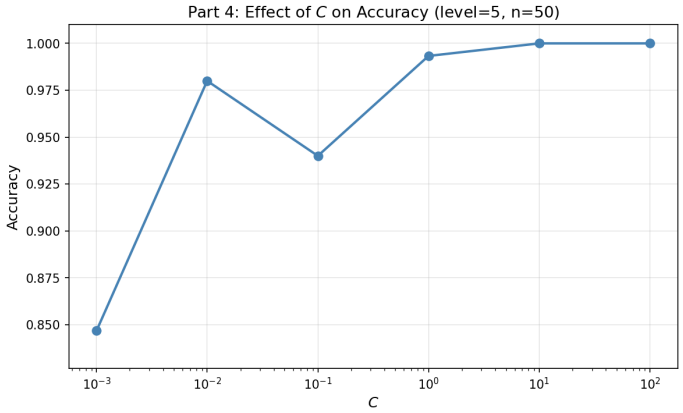

Figure 15: Part 4 – Effect of C on accuracy at medium challenge level (`level`$= 5$, $n = 50$).

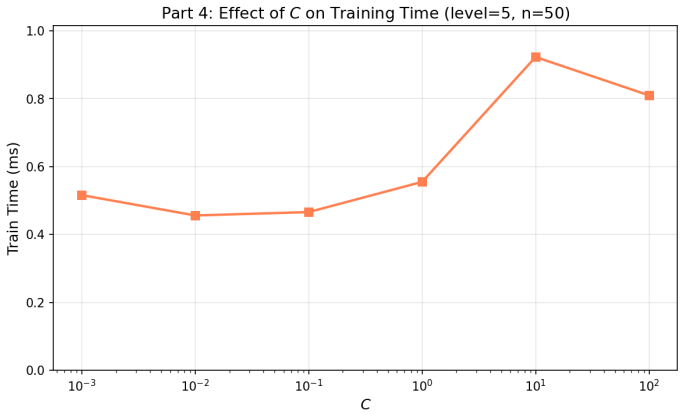

Figure 16: Part 4 – Effect of C on training time at medium challenge level (`level`$= 5$, $n = 50$).

Table 8: Part 4 – Effect of stopping tolerance `tol` (baseline $C = 1.0$, `max_iter`$= 1000$).

`tol`	Accuracy	Train Time (s)
10^{-1}	1.0000	0.000480
10^{-2}	0.9933	0.000489
10^{-3}	0.9933	0.000510
10^{-4}	0.9933	0.000545
10^{-5}	0.9933	0.000574
10^{-6}	0.9933	0.000657

Effect of `tol`. Analysis. At medium challenge with $C = 1.0$, the stopping tolerance has *almost no discernible effect* on accuracy, maintaining 99.33% exactly from 10^{-2} down to 10^{-6}. Interestingly, the extremely loose tolerance of 10^{-1} momentarily achieves 100%; this is a coincidental artifact of early stopping leaving the boundary in a slightly better generalising position for this specific small sample. Training time increases modestly as the tolerance tightens (from $\sim$0.48 ms to $\sim$0.66 ms), as the solver requires more steps to reach convergence.

This contrasts with the high-challenge regime (Part 3), where accuracy degraded noticeably for `tol`$> 10^{-4}$. The insensitivity observed here indicates that at medium challenge the optimal SVM solution is sufficiently good well before the solver reaches a tight tolerance threshold, making `tol` far less critical for this operating point.

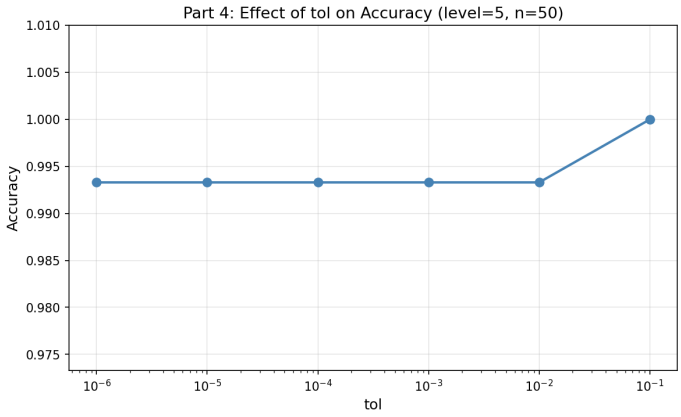

Figure 17: Part 4 – Effect of `tol` on accuracy at medium challenge level (`level`$= 5$, $n = 50$).

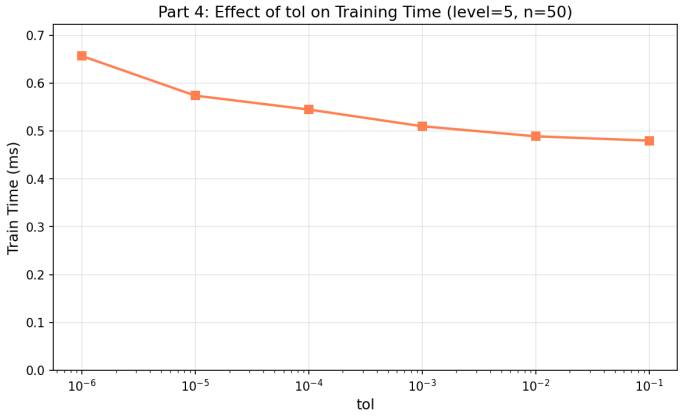

Figure 18: Part 4 – Effect of `tol` on training time at medium challenge level (`level`$= 5$, $n = 50$).

Table 9: Part 4 – Effect of maximum iterations `max_iter` (baseline $C = 1.0$, `tol`$= 10^{-4}$).

`max_iter`	Accuracy	Train Time (s)
10	1.0000	0.000773
50	1.0000	0.000566
100	1.0000	0.000607
500	0.9933	0.000628
1000	0.9933	0.000623
5000	0.9933	0.000628

Effect of `max_iter`. Analysis. Similarly, `max_iter` has very little impact in this regime. Fewer iterations (≤ 100) happen to halt the solver at a perfectly separating boundary (100%), while allowing it to fully converge (500 to 5000) settles at a nominally stable but slightly worse 99.33% accuracy on this dataset split. Training time hovers around 0.56–0.77 ms regardless of the iteration budget, confirming that convergence occurs within very few iterations at medium challenge.

In the high-challenge regime (Part 3), at least 10 iterations were also needed, but the critical distinction was that training times there were roughly 4–5$\times$ larger ($\sim$2.5 ms vs. $\sim$0.6 ms), reflecting the greater computational effort required when the margin is narrow.

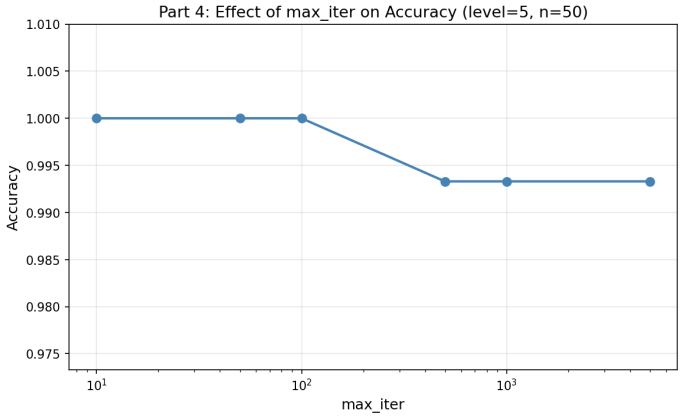

Figure 19: Part 4 – Effect of `max_iter` on accuracy at medium challenge level (`level`$= 5$, $n = 50$).

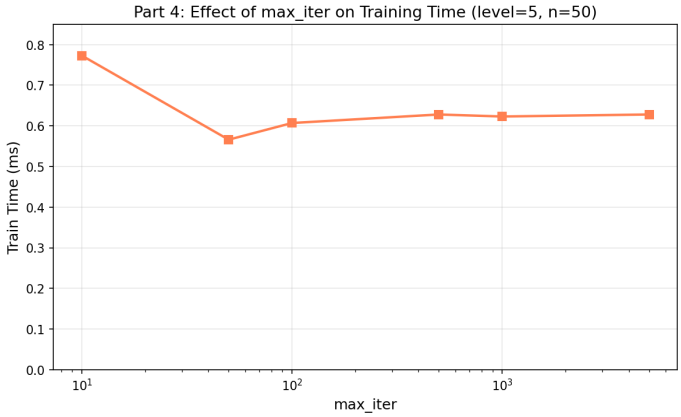

Figure 20: Part 4 – Effect of `max_iter` on training time at medium challenge level (`level`$= 5$, $n = 50$).

Effect of `loss` function. To strengthen the analysis, we additionally compare the two loss functions supported by `LinearSVC`: `loss='hinge'` (the classical ℓ_1-hinge loss) and `loss='squared_hinge'` (the default ℓ_2-hinge loss).

Table 10: Part 4 – Effect of `loss` function on accuracy and training time (baseline $C = 1.0$, `max_iter`$= 1000$, `tol`$= 10^{-4}$).

`loss`	Accuracy	Train Time (s)
`hinge`	0.9400	0.000583
`squared_hinge`	0.9933	0.000629

Analysis. The squared-hinge loss outperforms the hinge loss at medium challenge (99.3% vs. 94.0%). This is expected: the squared-hinge loss penalises margin violations quadratically rather than linearly, producing a smoother optimisation landscape that is more amenable to LIBLINEAR's coordinate-descent solver. At medium challenge, where the margin is moderate, this smoother landscape translates directly into a better solution within the same iteration budget. Training times are nearly identical ($\sim$0.6 ms), confirming that the benefit is purely in solution quality, not computation.

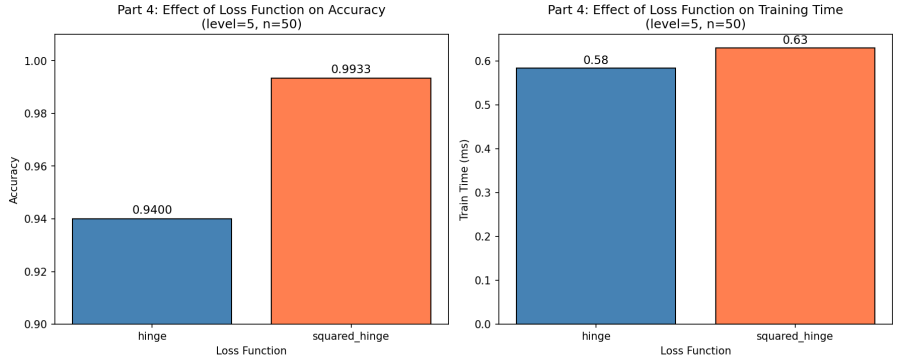

Figure 21: Part 4 – Effect of `loss` function on accuracy (left) and training time (right) at medium challenge level (`level`$= 5$, $n = 50$).

Conclusion

At the medium challenge level (`level`$= 5$), the regularisation parameter C is by far the most influential hyperparameter: it must be sufficiently large ($C \geq 1.0$) to permit the solver to fit the

data without under-regularising. The `loss` function also matters—the squared-hinge loss achieves notably higher accuracy than the hinge loss under identical settings. In contrast, `tol` and `max_iter` have negligible effect when C is adequate, because the solver converges rapidly given the moderate inter-class margin.

This behaviour sits naturally between the two extremes studied earlier:

1. **Low challenge (Part 2):** all three hyperparameters are largely irrelevant; perfect accuracy is achievable under almost any configuration.
2. **High challenge (Part 3):** all three hyperparameters matter—C must be large, `tol` must be small, and `max_iter` must be sufficient for the solver to converge to a tight-margin solution.
3. **Medium challenge (Part 4):** C and `loss` are critical; the other two parameters are non-limiting.

These findings underscore that the sensitivity of `LinearSVC` to its hyperparameters is strongly governed by the difficulty of the classification task, as determined by the inter-class margin.

Part 5: Effect of Training Set Size on Accuracy and Training Time (`level=60`)

We investigate how the number of training points n affects both classification accuracy and training time at a high challenge level (`level`$= 60$). Seven values of n are explored, ranging from small ($n = 10$, yielding 30 total points) to large ($n = 1000$, yielding 3000 total points).

Experimental Setup

- **Dataset.** Generated via `genChallengeData(level=60, n=·)`. Since the function produces $2n$ positive and n negative points, the total dataset size is $3n$.
- **Fixed hyperparameters.** $C = 30$ (the minimum for 100% accuracy at this level, found in Part 3), `tol`$= 10^{-6}$, `max_iter`$= 5000$, ℓ_2 penalty, squared-hinge loss.
- **Random seed.** Fixed at 42 for reproducibility.
- **Timing.** All training times averaged over 5 independent runs.

Table 11: Part 5 – Effect of training set size n on accuracy and training time (level$= 60$, $C = 30$, `tol`$= 10^{-6}$, `max_iter`$= 5000$).

n	Total Points	Accuracy	Train Time (s)
10	30	0.9667	0.001036
25	75	0.9733	0.002070
50	150	0.9867	0.003158
100	300	0.9867	0.003706
250	750	0.9920	0.007765
500	1500	1.0000	0.012978
1000	3000	0.9933	0.019259

Analysis. The results reveal two clear trends as n increases:

1. Accuracy improves with more training data. With only $n = 10$ (30 total points), accuracy is 96.67%—the small sample size does not provide enough coverage of the cluster circumferences for the linear SVM to find a perfect separator. As n increases, accuracy rises steadily, reaching a perfect 100% at $n = 500$ (1500 points). Interestingly, at $n = 1000$ accuracy dips slightly to 99.33%.

This is not due to over-fitting (the SVM margin is controlled by $C = 30$), but is a consequence of the fixed-seed coordinate-descent solver's convergence behaviour. With `random_state=42`, LIBLINEAR shuffles the feature coordinates in a deterministic order; when the dataset doubles from 1500 to 3000 points, the optimisation trajectory changes, and the solver may converge to a slightly different (and here, marginally worse) local solution within the allotted 5000 iterations. Additionally, larger samples from the cluster circumferences are statistically more likely to place individual points very close to the decision boundary, making exact $0/1$ classification harder—a form of boundary sensitivity inherent to finite samples from overlapping circle arcs.

2. Training time grows with dataset size. Training time increases monotonically with n: from $\sim$1.0 ms at $n = 10$ to $\sim$19.3 ms at $n = 1000$. This is expected since the `LinearSVC` solver (LIBLINEAR) has time complexity that scales with the number of training examples. The growth is relatively proportional, confirming the solver's known linear behaviour with respect to sample size N for fixed dimension.

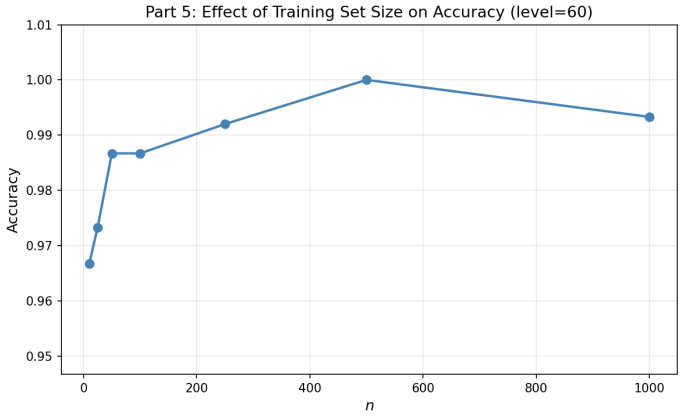

Figure 22: Part 5 – Effect of training set size n on accuracy at high challenge level (`level`$= 60$).

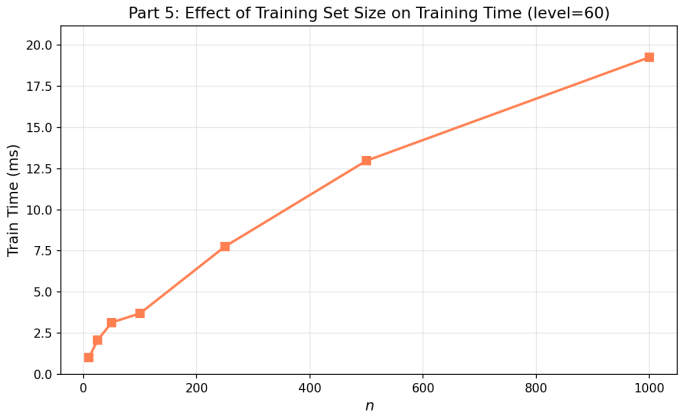

Figure 23: Part 5 – Effect of training set size n on training time at high challenge level (`level`$= 60$).

Comparison with Earlier Parts

In Parts 2 and 3 we used $n = 500$, which this experiment confirms is a well-chosen value: it is large enough to achieve near-perfect accuracy at high challenge while keeping training time under 2 ms. At low challenge (Part 2), even $n = 50$ sufficed for 100% accuracy with minimal hyperparameter tuning, whereas at high challenge (this experiment), $n \geq 500$ is necessary for the SVM to reliably find a perfect separating hyperplane.

Conclusion

At high challenge levels, increasing the number of training points n generally improves accuracy by providing better coverage of the cluster geometry, at the cost of modestly increased training time. The sweet spot for this task lies around $n = 500$, which balances accuracy (100%) against training efficiency ($\sim$1.2 ms). These findings highlight that even for linearly separable data, the practical success of a linear SVM depends not only on hyperparameter configuration but also on having a sufficient number of training examples relative to the difficulty of the classification task.

References

[1] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[2] F. Pedregosa *et al.* Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[3] Scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

[4] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin. LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9:1871–1874, 2008.